

DESENVOLVEDOR DE SISTEMAS

LINGUAGEM DE PROGRAMAÇÃO

PROFESSOR MICHALES CAMURÇA





Tipos Sequenciais em Python

◆ Strings

◆ Listas

◆ Tuplas

◆ Range

O que são?

◆ Estruturas que armazenam coleções ordenadas

◆ Permitem acesso por índice

1	2
3	4

◆ Alguns são mutáveis, outros não 

Índices

| | | |---|---| | 1 | 2 | | 3 | 4 | Acesso por posição

- Índices começam em 0
- Índices negativos percorrem de trás

→

←

EXEMPLO:

```
nome = "Python"  
print(nome[0]) # P  
print(nome[-1]) # n
```

Strings

Sequência de caracteres

- Imutáveis 
- Usadas para textos 

EXEMPLO:

```
texto = "Aprendendo Python"  
print(len(texto))    # tamanho  
print(texto.upper()) # maiúsculas  
print(texto[0:10])   # fatiamento
```

Listas

Coleção mutável

- Armazena diferentes tipos 
- Permite alterações 

EXEMPLO:

```
numeros = [10, 20, 30]  
numeros.append(40)  
numeros[1] = 25  
print(numeros) # [10, 25, 30, 40]
```

Tuplas

Coleção imutável

- Protege os dados 
- Mais rápida que listas 

EXEMPLO:

```
cores = ("azul", "verde", "vermelho")  
print(cores[0]) # azul  
print(cores[2]) # vermelho
```


Operações Úteis

Comuns em sequências

- `in` → verificar presença 
- `len()` → tamanho 
- `min()` / `max()` → menor e maior  

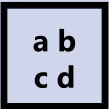
EXEMPLO:

```
lista = [5, 8, 2]  
print(8 in lista)  # True  
print(len(lista))  # 3
```

Comparação

Resumo dos tipos

- `in` → verificar presença 
- `len()` → tamanho 
- `min()` / `max()` → menor e maior  

Tipo	Mutável?	Exemplo
 String	Não	"Python"
 Lista	Sim	[1, 2, 3]
 Tupla	Não	(1, 2, 3)

Listas em Python

Tipos Sequenciais – Listas

Estruturas de dados dinâmicas em Python

Introdução às Listas

 O que são?

- Coleções ordenadas de elementos
- Podem conter tipos diferentes
- Mutáveis (podem ser alteradas)
- Definidas com colchetes []

EXEMPLO:

```
minha_lista = [10, "Python", 3.14, True]
```

Acesso aos Elementos

| | | |---|---| | 1 | 2 | | 3 | 4 | Índices e fatiamento

- Primeiro elemento: índice 0
- Último: índice -1
- Fatiamento: [início:fim]

EXEMPLO:

```
numeros = [5, 10, 15, 20]
print(numeros[0])    # 5
print(numeros[-1])   # 20
print(numeros[1:3])  # [10, 15]
```

Modificando Listas

Adicionar, alterar e remover

EXEMPLO:

```
frutas = ["maçã", "banana", "uva"]
```

```
frutas.append("laranja")      # adiciona
frutas[1] = "pera"           # modifica
frutas.remove("uva")          # remove
print(frutas)                 # ['maçã', 'pera', 'laranja']
```

Operações Comuns

Funções úteis

- `len(lista)` → tamanho 
- `in` → verificar item 
- `sort()` → ordenar 
- `reverse()` → inverter 

EXEMPLO:

```
numeros = [3, 1, 4, 2]
print(len(numeros))      # 4
print(3 in numeros)      # True
numeros.sort()
print(numeros)           # [1, 2, 3, 4]
```

Iterando com for

Percorrendo listas

EXEMPLO:

```
animais = ["gato", "cachorro", "peixe"]
```

```
for a in animais:
```

```
    print(a)
```

 Saída:

gato

cachorro

peixe

Listas Aninhadas

Lista dentro de lista

EXEMPLO:

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matriz[0][1]) # 2  
print(matriz[2][2]) # 9
```

Métodos Avançados de Listas em Python

⚡ Métodos Avançados de Listas

◆ Manipulação e funcionalidades poderosas

🔑 **Inserindo em Posição Específica**

🔑 `insert(pos, item)`

Adiciona em um índice específico

EXEMPLO:

```
frutas = ["maçã", "uva", "pera"]  
frutas.insert(1, "banana")  
print(frutas) # ['maçã', 'banana', 'uva', 'pera']
```



Removendo Elementos



Métodos de remoção:

- `pop()` → remove último ou por índice
- `remove(item)` → remove pela primeira ocorrência

EXEMPLO:

```
lista = [1, 2, 3, 4]
lista.pop()          # remove último
lista.pop(1)         # remove índice 1
lista.remove(3)      # remove valor 3
print(lista)         # [1]
```

Contar e Localizar

Procurar em listas

- `count(item)` → quantidade de vezes que aparece
- `index(item)` → posição da primeira ocorrência

EXEMPLO:

```
numeros = [2, 4, 2, 8, 2]  
print(numeros.count(2)) # 3  
print(numeros.index(8)) # 3
```

Ordenação Avançada

Métodos para ordenar:

- `sort()` → ordena lista
- `reverse()` → inverte ordem
- `sorted(lista)` → cria nova lista ordenada

EXEMPLO:

```
valores = [7, 2, 9, 4]
valores.sort()
print(valores)           # [2, 4, 7, 9]
valores.reverse()
print(valores)           # [9, 7, 4, 2]
```

Copiando Listas

 Evite erros ao duplicar listas:

- `copy()` → cópia rasa
- `list(original)` → nova lista

EXEMPLO:

```
a = [1, 2, 3]
```

```
b = a.copy()
```

```
b.append(4)
```

```
print(a) # [1, 2, 3]
```

```
print(b) # [1, 2, 3, 4]
```

Compreensão de Listas (List Comprehension)

Forma concisa de criar listas

EXEMPLO:

```
# Quadrados de 0 a 9
```

```
quadrados = [x ** 2 for x in range(10)]
```

```
print(quadrados)
```

```
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```


DÚVIDAS? PERGUNTAS? ANGÚSTIAS? AFLIÇÕES?



Lista de Desafios – Aula 10

-  D056: Crie uma lista com 6 números.
-  D057: Mostre o maior e o menor número.
-  D058: Ordene a lista em ordem crescente.
-  D059: Percorra a lista e imprima apenas os pares.
-  D060: Crie uma lista com 10 números aleatórios.
-  D061: Mostre quantas vezes o número 5 aparece.
-  D062: Ordene a lista em ordem crescente e depois inversa.
-  D063: Crie uma nova lista apenas com os números pares (list comprehension).



CETAM
CENTRO DE EDUCAÇÃO
TECNOLOGICA DO AMAZONAS



AMAZONAS
GOVERNO DO ESTADO

TRABALHO QUE TRANSFORMA